Index는 내가 원하는 Record의 위치를 가리키고있는 date Structure를 말한다.

이 Index는 다음과 같은 Record로 구성된 하나의 File이다

Search-Key	Pointer

여기서 Search key는 찾을 대상을 구분하는 역할을 하고
Pointer는 해당 Record의 실제 위치를 가리키고 있다.

이를 RDBMS에서 Relation에 적용하면 각각의 Attribute는 Search Key의 역할을 할 수 있다.
따라서 이를 활용해 Index File을 만들고 이를 활용하면 데이터를 읽어오는 속도를 매우 높일 수 있다.

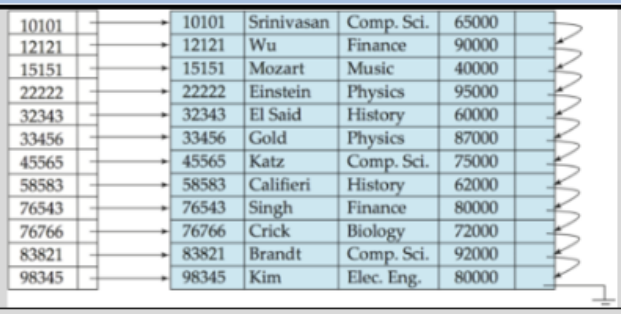
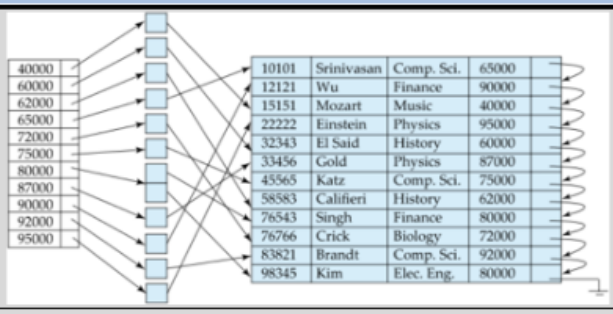
물론, Insertion이나 Deletion과 같은 DataBase의 Update에 있어 Index File도 추가적으로 Update해주어야 하므로 관리 비용이 존재하지만, Data Base의 주된 사용 목적은 Search이기 때문에 효율적인 RDBMS를 위해서는 Index를 적극적으로 활용해야한다.

> *index 평가요소*
>
> 1. Access Time
> : Data를 얼마나 빨리 찾을 수 있는지
>
> 2. Insertion Time
> : Data를 Insertion할 때 걸리는 시간
>
> 3. Deletion Time
> : Data를 Deletion할 때 걸리는 시간
>
> 4. Space Overhead
> : Index File의 크기를 얼마나 줄였는지

Index

1. Index Type

1) Primary VS Secondary

	Primary Index (=Clustering Index)	Secondary Index (=Non-Clustering Index)
예시		
정의	Sequential File에서의 정렬 기준과 Search Key의 대상이 동일한 Index File을 의미한다.	Sequential File에서의 정렬 기준과 Search Key의 대상이 다른 Index File을 의미한다.

(data file=sequential file)

	Primary Index	Secondary Index
정의	처음부터 정렬이 되어있는 Attribute에 대한 Index File	정렬되어 있지 않은 Attribute에 대한 Index File
값	Block의 위치	Block의 포인터 배열의 위치
개수	Table당 한 개	Table당 여러 개
Select속도	빠름	느림

	Primary Index	Secondary Index
I/M/D속도	느림	빠름

(I:Insert, M:Modify, D:Delete)

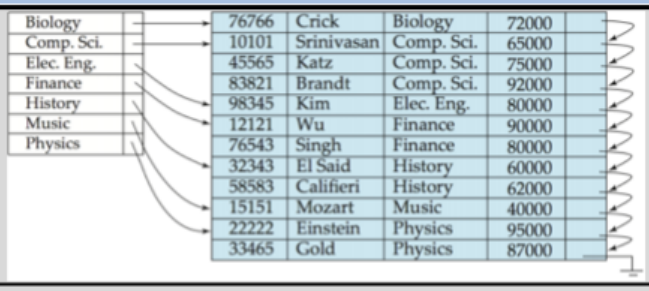
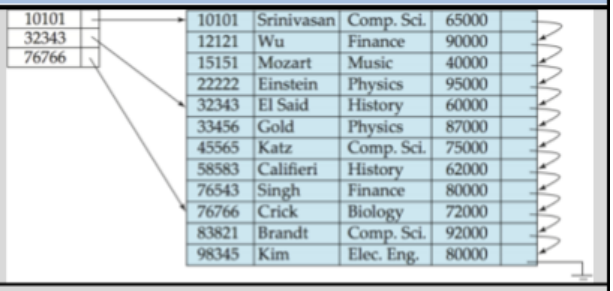
My SQL의 경우 Primary Key를 지정해주면 자동으로 해당 Attribute를 Primary Index로 하는 Index File을 생성한다.

Sequential Scan시 I/O Complexity

1. Primary Index의 경우
   : Data File과 정렬 순서가 일치하므로 Sparse Index를 활용해
   $O(\frac{N}{B})$의 I/O Complexity 가 가능하다.

2. Secondary Index의 경우
   : Data File과 정렬 순서가 다르기 때문에 Sparse Index를 활용할 수 없고
   $O(N)$의 I/O Complexity 가 발생하게 된다.

2) Dense VS Sparse

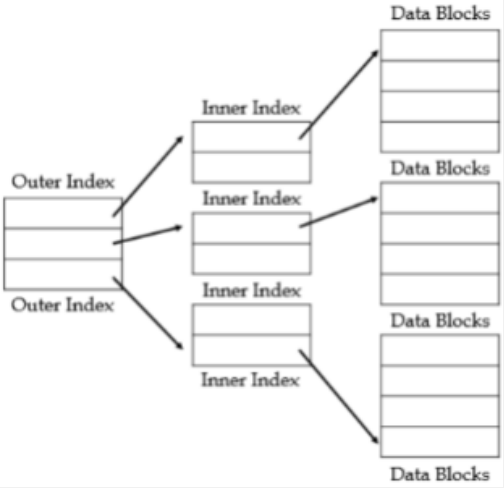
	Dense Index	Sparse Index
예시		
정의	Data File의 모든 Search Key를 표현하고 있는 Index File을 의미한다.	Data File의 몇몇 대표되는 Search Key만을 표현하고 있는 Index File을 의미한다.
참고	즉, 위와 같이 Dense Index의 Entry 개수는 실제 Data File의 Record의 개수와 일치해야 할 필요는 없다.	Sparse Index의 Entry는 Data File의 Block별로 대표되는 값을 정해 결정한다. 즉, $\frac{N}{B}$의 Entry 개수를 갖게 된다.
중요	**모든 Secondary Index는 Dense Index여야만 한다.**	**모든 Sparse Index는 Primary Index여야만 한다.**

(역은 성립하지 않는다.)

Sparse Index를 사용할 경우 Index File의 크기를 효과적으로 줄일 수 있다.
이는, Index File조차도 Memory에 올라갈 수 없을 정도로 클 때 해결책이 될 수 있다.

Multi Level Index

Sparse Index File을 층층이 쌓아 Multi Level Index File을 만들 수 있다.



(Inner Index: Primary Index of Data File)
(Outer Index: Sparse Index of Primary Index)

이때, 각 i번째 층은 $\frac{N(i-1)}{B(i-1)}$개의 Entry를 갖게된다.

즉, 각 층마다 Block의 수를 $\frac{1}{B}$씩 줄일 수 있게되고,
이를 통해 Data에 접근하기 위해 Search할 Block의 수를 효과적으로 줄일 수 있다.

---

4. Multi Level Index의 단점

Search만 하는 DB의 경우 Multi Level Index는 단점이 없는 구조가 되지만 Update가 발생할 경우 문제가 발생하게 된다.

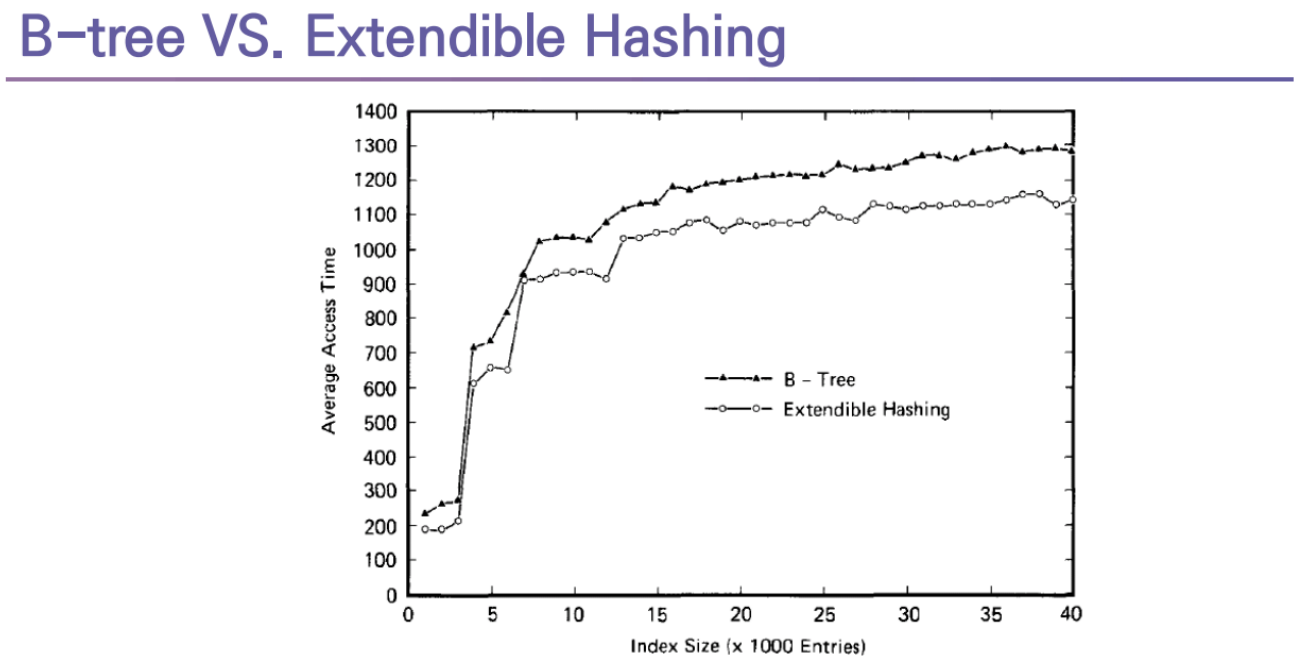
Data를 Delete/Insert할 때, Free Space가 많은 Data Block이 발생하게 된다.

즉, 대표값을 통해 Block Access횟수를 줄인다는 Sparse Index의 장점이 사라지게 된다.

따라서 주기적인 Reorganization이 필요하다.

---

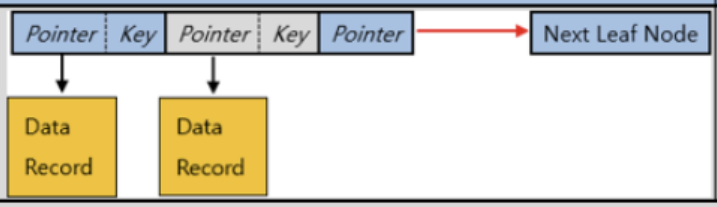
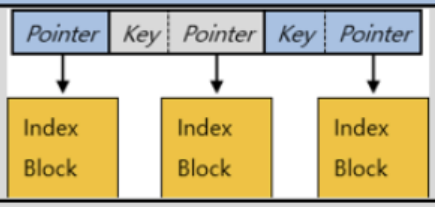
2. Index Algorithm Type



이제 Index File을 설계하는 방법들에 대해서 간단하게 살펴보도록 하자

1) Hash

2) B+ Tree Index

	Leaf Node	Non-Leaf Node
구조		
개념	1. Balanced Tree로 모든 Leaf Node의 Level이 같다. 2. Search Tree로써 Key를 기준으로 $Left\ Child\ Key < Key \le Right\ Child\ Key$의 규칙이 존재한다. 3. Leaf Node는 실제 값을, Non-Leaf Node는 다른 Index Block을 가리키도록 한다. **4. 모든 Node는 정해진 값 이상의 Pointer 수를 가져야 한다.** 5. 하나의 Node에는 하나의 Block이 할당된다.	
설명	**Key** : 찾을 Data에 대한 값이 들어있다. **Pointer** : Key에 해당하는 Record를 가리키고 있다. **마지막 Pointer** : 다음 Leaf Node를 가리키고 있다.	**Key** : 오른쪽 Pointer가 가리키는 Index Block의 Key의 최솟값이 들어있다. **Pointer** : $(\text{왼쪽}Key \le x < \text{오른쪽}Key)$인 x를 Key로 갖는 Block을 가리킨다.
참고	Search Key의 수: $\left[\left[\frac{B-1}{2}\right], B-1\right]$ Pointer의 수: $\left[\left[\frac{B-1}{2}\right]+1, B\right]$ 기준은 "Search Key의 수" 가 된다.	Search Key의 수: $\left[\left[\frac{B}{2}\right]-1, B-1\right]$ Pointer의 수: $\left[\left[\frac{B}{2}\right], B\right]$ 기준은 "Pointer의 수" 가 된다.

(B는 한 Node(Block)에 들어갈 수 있는 Pointer의 최대 개수를 의미한다.)
(Root Node에 대해서는 Pointer 수에 대한 기준이 $[2, B]$로 예외적으로 존재한다)

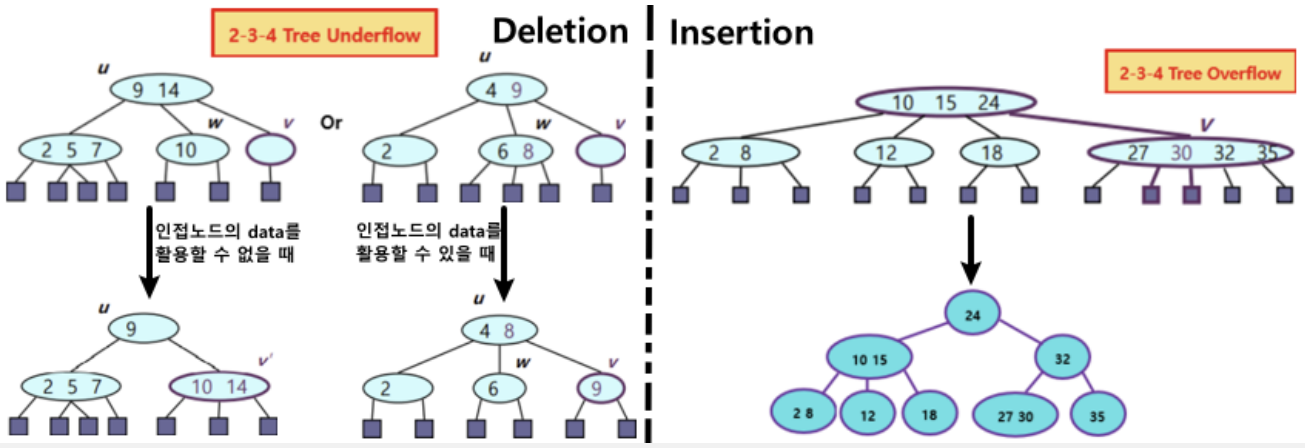
Search

1. **Point Query**
   - 방법
     : Search Tree임을 활용해 Leaf Node까지 이동하면 찾을 수 있다.
   - I/O Complexity
     : $O(log_B(N))$
     ($Height+1$ 번 Block에 접근하게 되기 때문)

2. **Range Query**
   - 방법
     : Point Query를 활용해 최솟값을 찾아 data를 Return하고,
     Leaf Node를 쭉 훑으면서 최대값이 나오기 전까지 Data를 Return한다.
   - I/O Complexity
     : $O(log_B(N))+\frac{T}{B}$
     ($Height+1+\frac{T}{B}$ 번 Block에 접근하게 되기 때문)

(T는 찾고자하는 Record의 개수)

Update



	Deletion	Insertion
Leaf Node	1. <Pointer, Key> 쌍을 삭제한다. (첫번째 쌍이 삭제된 경우 Parent의 값 변경) ("UnderFlow"가 발생하지 않으면 여기서 종료)	1. <Pointer, Key> 쌍을 추가한다. ("OverFlow"가 발생하지 않으면 여기서 종료)
	2. UnderFlow된 Node와 Sibling을 Merge한다. (C, C'를 합쳐 C로 만든다.)	*(현재 Node에는 B개의 Key가 존재한다.)* **2. OverFlow된 Node를 Key의 개수를 기준으로 Split한다.** ($C = \left[\frac{B}{2}\right]$, $C' = B - \left[\frac{B}{2}\right]$ 개의 **Key**를 가지도록 Split)
	3. C'의 Parent에서 $<Key, C'Pointer>$를 삭제한다.	3. Split후 새로 생긴 C'를 위해 $<c', C'Pointer>$ 쌍을 만든다.
	4. $C \rightarrow D$ 가 되도록 Pointer를 조정한다. *(D는 Split하기 전 Node의 Next Node)*	4. Parent Node에 Insert한다. *(Non-Leaf Node이므로 <key, pointer>)*
	5. Merge 후 OverFlow가 발생하면 다시 Split한다. (Insertion의 Split과 동일)	5. $C \rightarrow C' \rightarrow D$ 가 되도록 Pointer를 조정한다. *(D는 Split하기 전 Node의 Next Node)*
Non Leaf Node	1. <Key, Pointer> 쌍이 삭제된다. (첫번째 쌍이 삭제된 경우 Parent의 값 변경) ("UnderFlow"가 발생하지 않으면 여기서 종료)	1. <Key, Pointer> 쌍이 추가된다. ("OverFlow"가 발생하지 않으면 여기서 종료)
	2. UnderFlow된 Node와 Sibling을 Merge한다. (C, C'를 합쳐 C로 만든다.)	*(현재 Node에 B+1개의 Pointer가 존재한다.)* **2. OverFlow된 Node를 Pointer의 개수를 기준으로 Split한다.** ($C = \left[\frac{B}{2}\right]$, $C' = B - \left[\frac{B}{2}\right]$ 개의 **Pointer**를 가지도록 Split)
	3. C'의 Parent에서 $<Key, C'Pointer>$를 삭제한다.	3. Split후 중간에 남은 Key에 대해, $<C\ Pointer, Key, C'\ Pointer>$쌍을 만든다.
	4. Merge 후 OverFlow가 발생하면 다시 Split한다. (Insertion의 Split과 동일)	4. 3번에서 만든 쌍을 Parent Node에 추가한다.
Cost	우선 삭제한 Record를 찾는다: $O(\log_B N)$ 최악의 경우 Height만큼 Merge한다: $1 \times O(\log_B N)$ 즉, $O(\log_B N)$의 I/O Compleixty를 갖는다.	우선 넣을 자리를 찾는다: $O(\log_B N)$ 최악의 경우 Height만큼 Split한다: $1 \times O(\log_B N)$ 즉, $O(\log_B N)$의 I/O Compleixty를 갖는다.

1. Leaf Node에 Insertion/Deletion을 수행하고 이것이 Propagate될 경우,
   Non Leaf Node에서의 Insertion/Deletion을 수행한다.
2. Root가 Split되는 경우는 Height가 증가할 때 뿐이다.*(역도 성립)*
3. OverFlow와 UnderFlow는 Parent로 Propagate될 수 있다.

참고

B+ Tree는 Update에 Cost가 발생한다는 단점이 있었다.

> *이때, Primary Index는 어쩔 수 없지만 B+Tree로 구현된 Secondary Index는*
> *Data 자체가 아닌 Primary Index를 Indexing 하므로써, Update의 Cost를 없앨 수 있다.*

Multi Key Access

여러 Index File을 활용해야 하는 경우 다음과 같은 방법을 사용할 수 있다.

1. 먼저 하나의 Index File로 Data를 검색한다. 그리고 그 결과에 대해 나머지 조건을 Sequential하게 검사한다.
2. 각각의 Index File로 Data를 검색한다. 그리고 그 결과의 교집합을 구한다.
3. Index File의 Dememsion을 늘려 다시 만든다. (학부생 수준 X)

MySQL

1. STUDENT Table에 대해서
   ${name, sex, dno}$를 index로 하는 BTREE Index File생성

```
CREATE INDEX
   student_index ON STUDENT(name, sex) USING BTREE;
```

2. STUDENT Table을 대상으로 생성한 Index File분석

```
SHOW INDEX FROM STUDENT
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	...
student	1	student_index	1	name	A	1030	...
student	1	student_index	2	sex	A	2038	...

- Table: table 이름
- Non_unique: 인덱스의 중복값 가능 여부(1→가능, 0→불가능)
- Key_name: 인덱스의 이름
- Seq_in_index: Multi Column Index일 경우 필드의 순서
- Column_name: 해당 Column의 이름
- Collation: 인덱스 정렬 방식(A→오름차순, D→내림차순)
- Cardinality: 해당 컬럼의 상대적인 중복도 수치
  - Cardinality낮음 → 중복도 높음
  - Cardinality높음 → 중복도 낮음

> Cardinality가 높은 Column에서 낮은 Column순서대로 Index를 생성하는 것이 효율적임

3. Query 실행 계획

```
Explain + (SQL Query)
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student	NULL	ref	student_index	student_index	203	const	24278	10.00	Using index condition

- rows: 쿼리 전체에 의해 접근하는 행의 수

- filtered: 필터링 되고 남은 레코드의 비율(0~100)
  (Join 이나 Where 절에서 얼마나 많은 행이 필터링 되었는지 알 수 있음)